中山大学计算机学院本科生实验报告

（2024 学年第 2 学期）

课程名称：并行算法　　批改人：

实验	字符串模式匹配	专业（方向）	信息与计算科学
学号	22336049	姓名	陈日康
Email	chenih5@mail2.sysu.edu.cn	完成日期	2025 年 6 月 18 日

1. 实验内容

使用 C++ 语言实现两种并行字符串模式匹配算法，分别为并行 KMP 算法和并行近似串匹配算法，旨在掌握并行编程的基本方法，并通过实验分析其在不同数据规模下的性能提升效果。实现的算法需在逻辑上保证正确性，能够处理主串长度达到 10^7 数量级的数据。程序必须采用多线程或多进程实现，代码应支持不同数量的线程进行测试，可使用 C++11/17 线程库、OpenMP、MPI 或 CUDA 等并行编程工具，实验过程中需明确说明任务划分、数据分块、线程管理等并行化策略。实验至少测试 5 组不同规模的随机数据（如 10^5、10^6、10^7 等），并对负载均衡情况和两种算法的性能差异进行对比分析。

2. 算法原理

(1). 并行 KMP 算法

并行 KMP 算法的基本原理是通过预处理模式串，构造其部分匹配表（即前缀函数表），用于在匹配过程中快速回退，从而避免重复比较，整体时间复杂度为 O(n+m)，其中 n 为主串长度，m 为模式串长度。在并行实现中，将主串划分为若干个重叠区间，每个区间由独立线程执行 KMP 匹配。之所以需要重叠，是为了避免模式串跨线程边界时遗漏可能的匹配结果。每个线程根据给定的偏移量执行本地匹配任务，并将匹配位置回传主线程统一汇总。由于前缀表是静态构造且只依赖于模式串，可被所有线程共享，因此线程间几乎不存在通信开销，具有良好的可扩展性和负载均衡性能。

(2). 并行近似串匹配算法

并行近似字符串匹配算法则处理允许部分字符差异的匹配情形，适用于有编辑误差的数据搜索任务。其核心是对主串的每一个可能起始位置，滑动一个与模式串等长的窗口，并计算该窗口子串与模式串之间的最小编辑距离。编辑距离采用动态规划方式求解，时间复杂度为 O(mk)，其中 k 是最大允许编辑操作数。算法引入了带界编辑距离策略，即在动态规划过程中一旦发现某一行的最小值超过 k，则可提前终止计算，避免不必要的开销。在并行实现中，主串同样被划分为多个任务区间，由不同线程并行滑窗比较，每个线程独立记录符合条件的匹配位置。该方法无需线程间通信，结果统一合并后可得全局匹配列表。相比于并行 KMP 算法，近似匹配计算量更大，但带界优化显著提升了在误差控制下的搜索效率。

3. 关键代码解析

(1). 并行 KMP 算法（parallel_kmp.cpp）

(i). vector<int> build_kmp_table

```cpp
vector<int> build_kmp_table(const string& pattern) {
    int m = pattern.size();
    vector<int> prefix(m, 0);
    for (int i=1, j=0; i<m;) {
        if (pattern[i] == pattern[j])
            prefix[i++] = ++j;
        else if (j>0)
            j = prefix[j-1];
        else
            prefix[i++] = 0;
    }
    return prefix;
}
```

函数构建 KMP 算法的“部分匹配表”（前缀函数），用于加速匹配过程中失配后的跳转。通过构造 prefix[i] 表示 pattern[0...i] 的最长前缀后缀长度。在匹配失败时，直接跳转至 j = prefix[j-1] 位置，避免重复比较。

(ii). vector<int> kmp_search

```cpp
vector<int> kmp_search(const string& text, const string& pattern, const vector<int>& prefix, int offset=0) {
    vector<int> result;
    int n=text.size(), m=pattern.size();
    for (int i=0, j=0; i<n;) {
        if (text[i] == pattern[j]) {
            ++i;
            ++j;
            if (j==m) {
                result.push_back(i-j+offset);
                j = prefix[j-1];
            }
        }
        else if (j>0)
            j = prefix[j-1];
        else
            ++i;
    }
    return result;
}
```

函数基于 KMP 算法在主串 text 中查找所有模式串 pattern 出现的位置。利用前缀表跳过已匹配字符，避免回溯主串，匹配成功后记录匹配位置，加上 offset 以恢复全局坐标。

(iii). void parallel_kmp_worker

```cpp
void parallel_kmp_worker(const string& text, const string& pattern, const vector<int>& prefix, int start,
                         int end, int offset, vector<int>& local_result, vector<bool>& thread_called, int tid) {
    thread_called[tid] = true;
    string segment = text.substr(start, end-start);
    local_result = kmp_search(segment, pattern, prefix, offset);
}
```

函数是并行 KMP 算法中的核心线程函数，主要是在给定的子区间内执行一次局部的 KMP 匹配操作。text.substr(start, end-start) 提取分段主串线程结果保存在 local_resultthread_called[tid] 用于验证线程是否被调用成功。

(iv). void parallel_kmp_worker

```
void parallel_kmp(const string& text, const string& pattern, int num_threads,
                  vector<int>& match_positions, vector<bool>& thread_called) {
    int n=text.size(), m=pattern.size();
    vector<int> prefix = build_kmp_table(pattern);
    vector<thread> threads;
    vector<vector<int>> thread_results(num_threads);
    thread_called.resize(num_threads, false);
    int chunk_size = n/num_threads;

    for (int i=0; i<num_threads; ++i) {
        int start = i * chunk_size;
        int end = (i == num_threads-1) ? n : (i+1)*chunk_size+m-1;
        if (end>n)
            end = n;
        threads.emplace_back(parallel_kmp_worker, cref(text), cref(pattern), cref(prefix), start, end, start,
                             ref(thread_results[i]), ref(thread_called), i);
    }

    for (auto& t : threads)
        t.join();

    match_positions.clear();
    for (const auto& vec:thread_results)
        match_positions.insert(match_positions.end(), vec.begin(), vec.end());
    sort(match_positions.begin(), match_positions.end());
}
```

函数是并行 KMP 算法中的核心线程函数，每个线程在主串的一个指定子区间上独立执行局部 KMP 匹配操作。它接收一段主串的范围 [start, end)，利用 text.substr(start, end - start) 截取出当前线程负责的子串片段，然后调用 kmp_search 执行匹配操作。由于 kmp_search 接收了一个 offset 参数，能够将匹配位置转换为相对于整个主串的全局坐标，因此每个线程可以放心处理自己负责的子串段，无需关心全局位置的调整问题。

(2). 并行近似串匹配算法（parallel_approx_match.cpp）

(i). int bounded_edit_distance

```
int bounded_edit_distance(const string& text, const string& pattern, int k) {
    int n=text.size(), m=pattern.size();
    vector<vector<int>> dp(2, vector<int>(m + 1));
    for (int j=0; j<=m; ++j)
        dp[0][j] = j;

    for (int i=1; i<=n; ++i) {
        dp[i%2][0] = i;
        int min_val = dp[i%2][0];
        for (int j=1; j<=m; ++j) {
            if (text[i-1] == pattern[j-1])
                dp[i%2][j] = dp[(i-1) % 2][j-1];
            else
                dp[i%2][j] = 1 + min({ dp[(i-1) % 2][j], dp[i%2][j-1], dp[(i-1) % 2][j-1] });
            min_val = min(min_val, dp[i % 2][j]);
        }
        if (min_val>k)
            return k + 1;
    }
    return dp[n%2][m];
}
```

函数用于计算两个字符串之间的编辑距离，并在距离超过指定上限 k 时提早终止计算。实现方式采用滚动数组优化的动态规划，其中 dp[i%2][j] 表示将主串前 i 个字符变换为模式串前 j 个字符的最小操作数（包括插入、删除、替换）。在每一轮计算中，一旦发现当前行的最小编辑距离已经大于 k，就可以立即返回 k+1，表示无需继续计算，从而大大节省了时间开销。

(ii). void approximate_match_worker

```
void approximate_match_worker(const string& text, const string& pattern, int k, int start, int end,
                              vector<int>& local_result, vector<bool>& thread_called, int tid) {
    thread_called[tid] = true;
    int m = pattern.size();
    for (int i=start; i<=end-m; ++i) {
        int dist = bounded_edit_distance(text.substr(i, m), pattern, k);
        if (dist<=k)
            local_result.push_back(i);
    }
}
```

函数负责在指定主串区间 [start, end) 上执行近似匹配。每个线程会从 start 位置开始，逐个尝试提取长度为 pattern.size() 的主串片段，并调用 bounded_edit_distance 判断该子串与模式串的编辑距离是否不超过 k。如果满足条件则将匹配起始位置加入本线程的结果集合 local_result。此外，线程启动时会将 thread_called[tid] 置为 true，用于实验阶段验证线程是否被成功调度和执行。

(iii). void parallel_approx_match

```
void parallel_approx_match(const string& text, const string& pattern, int k, int num_threads, vector<int>& match_positions,
                           vector<vector<int>>& thread_results, vector<bool>& thread_called) {
    int n=text.size(), m=pattern.size();
    int chunk_size = n/num_threads;
    vector<thread> threads;
    thread_results.resize(num_threads);
    thread_called.resize(num_threads, false);

    for (int i=0; i<num_threads; ++i) {
        int start = i*chunk_size;
        int end = (i == num_threads-1) ? n : (i+1) * chunk_size+m-1;
        if (end>n)
            end = n;
        threads.emplace_back(approximate_match_worker, cref(text), cref(pattern), k, start, end,
                             ref(thread_results[i]), ref(thread_called), i);
    }

    for (auto& t:threads)
        t.join();

    match_positions.clear();
    for (auto& vec:thread_results)
        match_positions.insert(match_positions.end(), vec.begin(), vec.end());
    sort(match_positions.begin(), match_positions.end());
}
```

函数负责启动多个线程并行执行近似字符串匹配。首先根据线程数将主串划分为若干段，每段的起始和结束位置会预留 pattern.size()-1 个字符用于处理边界重叠，然后为每段区间启动一个线程调用 approximate_match_worker 处理，所有线程的匹配结果存储在 thread_results 中。所有线程结束后，将每个线程的局部结果合并、排序并输出为最终的匹配位置。

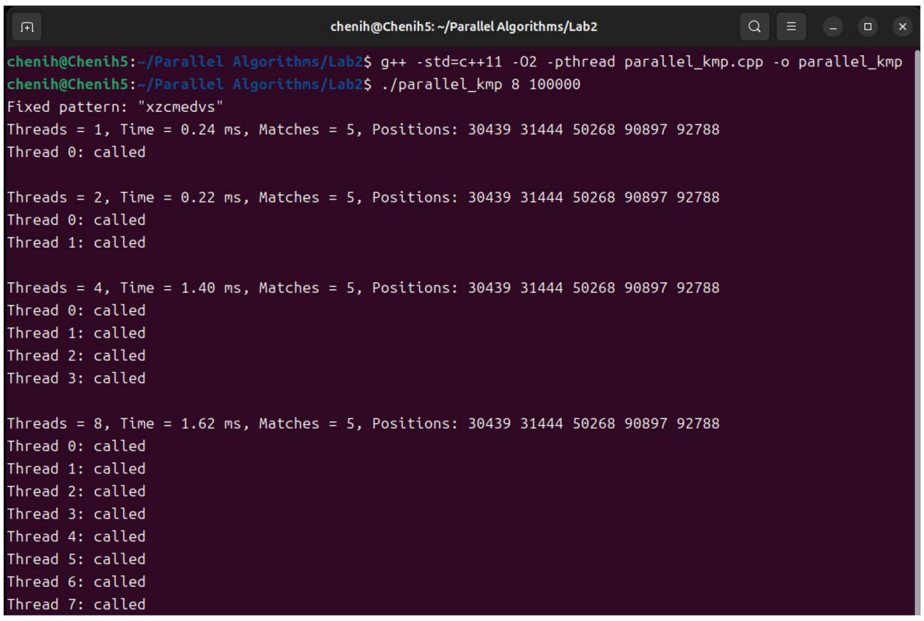
4. 实验结果

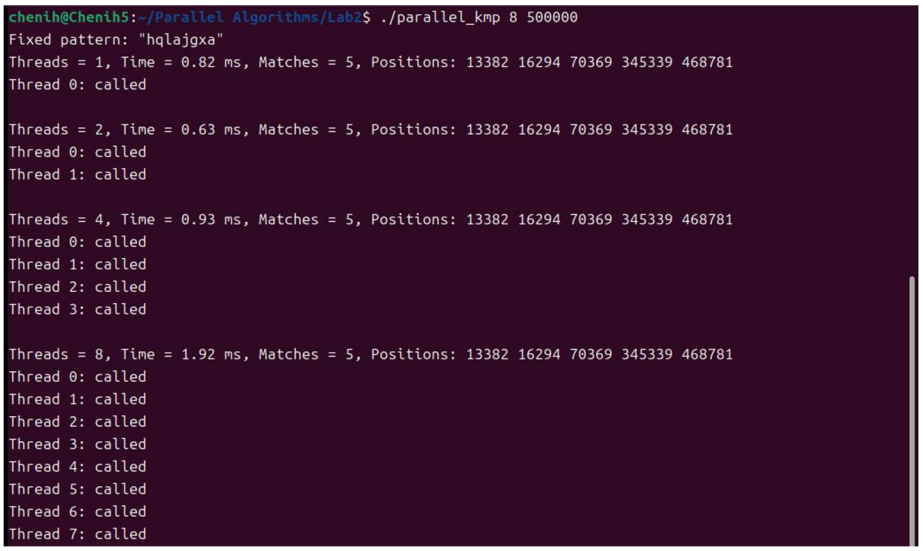
(1). 并行 kmp 算法（parallel_kmp.cpp）

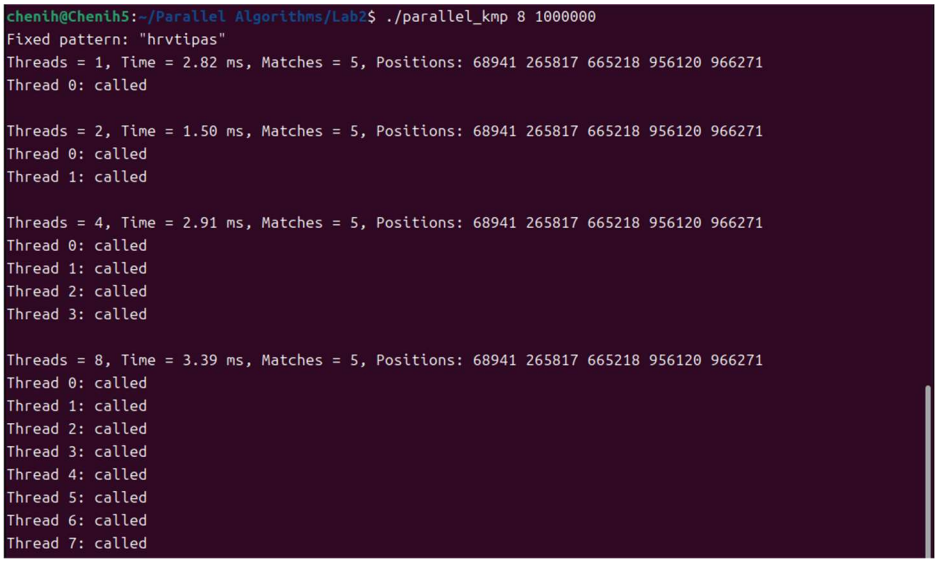
输入格式：`g++ -std=c++11 -O2 -pthread parallel_kmp.cpp -o parallel_kmp`
`./parallel_kmp <pattern_length> <text_length>`

输出格式：每组线程数对应一行运行结果，包含匹配用时、匹配数量及前若干个匹配位置（最多前 5 个位置，超出部分以...省略），随后逐行列出各线程是否被调用。测试的主串长度分别为 100000（10^5）、

500000（$5*10^5$）、1000000（10^6）、5000000（$5*10^6$）、10000000（10^7）、50000000（$5*10^7$）和 100000000（10^8），模式串的长度为 8，线程数分别为 1、2、4、8，模式串插入位置的个数设置为 5：







```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./parallel_kmp 8 5000000
Fixed pattern: "jawzymqk"
Threads = 1, Time = 11.68 ms, Matches = 5, Positions: 670784 2825994 2925116 3237205 4333368
Thread 0: called

Threads = 2, Time = 9.14 ms, Matches = 5, Positions: 670784 2825994 2925116 3237205 4333368
Thread 0: called
Thread 1: called

Threads = 4, Time = 10.58 ms, Matches = 5, Positions: 670784 2825994 2925116 3237205 4333368
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 13.22 ms, Matches = 5, Positions: 670784 2825994 2925116 3237205 4333368
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./parallel_kmp 8 10000000
Fixed pattern: "nxfzaepm"
Threads = 1, Time = 14.61 ms, Matches = 5, Positions: 461144 6514880 6632618 6846424 9929719
Thread 0: called

Threads = 2, Time = 7.86 ms, Matches = 5, Positions: 461144 6514880 6632618 6846424 9929719
Thread 0: called
Thread 1: called

Threads = 4, Time = 8.28 ms, Matches = 5, Positions: 461144 6514880 6632618 6846424 9929719
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 14.65 ms, Matches = 5, Positions: 461144 6514880 6632618 6846424 9929719
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./parallel_kmp 8 50000000
Fixed pattern: "qxtbkpll"
Threads = 1, Time = 135.42 ms, Matches = 5, Positions: 15134015 15557907 31430089 38190579 39361502
Thread 0: called

Threads = 2, Time = 104.69 ms, Matches = 5, Positions: 15134015 15557907 31430089 38190579 39361502
Thread 0: called
Thread 1: called

Threads = 4, Time = 36.94 ms, Matches = 5, Positions: 15134015 15557907 31430089 38190579 39361502
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 85.05 ms, Matches = 5, Positions: 15134015 15557907 31430089 38190579 39361502
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./parallel_kmp 8 100000000
Fixed pattern: "vnxawooq"
Threads = 1, Time = 435.78 ms, Matches = 5, Positions: 2737908 15189214 45322870 45667028 86452367
Thread 0: called

Threads = 2, Time = 116.65 ms, Matches = 5, Positions: 2737908 15189214 45322870 45667028 86452367
Thread 0: called
Thread 1: called

Threads = 4, Time = 139.60 ms, Matches = 5, Positions: 2737908 15189214 45322870 45667028 86452367
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 80.35 ms, Matches = 5, Positions: 2737908 15189214 45322870 45667028 86452367
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

(2). 并行近似串匹配算法（parallel_approx_match.cpp）

输入格式：`g++ -std=c++11 -O2 -pthread parallel_approx.match -o approx_match`
`./parallel_kmp <pattern_length> <text_length> <k>`

输出格式：每组线程数对应一行运行结果，包含匹配用时、匹配数量及前若干个匹配位置（如果匹配位置较多，位置列表仅显示前 5 项，末尾用...表示省略），随后逐行列出各线程是否被调用。主串长度分别为 100000（10^5）、500000（$5*10^5$）、1000000（10^6）、5000000（$5*10^6$）、10000000（10^7）、50000000（$5*10^7$）和 100000000（10^8），模式串的长度为 8，线程数分别为 1、2、4、8，设定最大边值距离 k=2：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab2
chenih@Chenih5:~/Parallel Algorithms/Lab2$ g++ -std=c++11 -O2 -pthread parallel_approx_match.cpp -o approx_match
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./approx_match 8 100000 2
Fixed pattern: "gwvlktzq"
Threads = 1, Time = 9.41 ms, Matches = 15, Positions: 29010 29011 29012 48882 48883 ...
Thread 0: called

Threads = 2, Time = 9.66 ms, Matches = 15, Positions: 29010 29011 29012 48882 48883 ...
Thread 0: called
Thread 1: called

Threads = 4, Time = 6.97 ms, Matches = 15, Positions: 29010 29011 29012 48882 48883 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 12.03 ms, Matches = 15, Positions: 29010 29011 29012 48882 48883 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called

chenih@Chenih5:~/Parallel Algorithms/Lab2$
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./approx_match 8 500000 2
Fixed pattern: "vsbxxchc"
Threads = 1, Time = 53.98 ms, Matches = 15, Positions: 269234 269235 269236 285737 285738 ...
Thread 0: called

Threads = 2, Time = 49.00 ms, Matches = 15, Positions: 269234 269235 269236 285737 285738 ...
Thread 0: called
Thread 1: called

Threads = 4, Time = 48.52 ms, Matches = 15, Positions: 269234 269235 269236 285737 285738 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 55.68 ms, Matches = 15, Positions: 269234 269235 269236 285737 285738 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./approx_match 8 1000000 2
Fixed pattern: "mmsjwfwg"
Threads = 1, Time = 91.35 ms, Matches = 15, Positions: 249400 249401 249402 342372 342373 ...
Thread 0: called

Threads = 2, Time = 57.89 ms, Matches = 15, Positions: 249400 249401 249402 342372 342373 ...
Thread 0: called
Thread 1: called

Threads = 4, Time = 62.65 ms, Matches = 15, Positions: 249400 249401 249402 342372 342373 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 59.35 ms, Matches = 15, Positions: 249400 249401 249402 342372 342373 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./approx_match 8 5000000 2
Fixed pattern: "gbkvxnud"
Threads = 1, Time = 445.17 ms, Matches = 15, Positions: 408789 408790 408791 446114 446115 ...
Thread 0: called

Threads = 2, Time = 211.06 ms, Matches = 15, Positions: 408789 408790 408791 446114 446115 ...
Thread 0: called
Thread 1: called

Threads = 4, Time = 252.59 ms, Matches = 15, Positions: 408789 408790 408791 446114 446115 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 241.41 ms, Matches = 15, Positions: 408789 408790 408791 446114 446115 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./approx_match 8 10000000 2
Fixed pattern: "jlcmzsop"
Threads = 1, Time = 874.12 ms, Matches = 15, Positions: 2736994 2736995 2736996 2953703 2953704 ...
Thread 0: called

Threads = 2, Time = 605.91 ms, Matches = 15, Positions: 2736994 2736995 2736996 2953703 2953704 ...
Thread 0: called
Thread 1: called

Threads = 4, Time = 572.32 ms, Matches = 15, Positions: 2736994 2736995 2736996 2953703 2953704 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 516.76 ms, Matches = 15, Positions: 2736994 2736995 2736996 2953703 2953704 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./approx_match 8 50000000 2
Fixed pattern: "qlikybhl"
Threads = 1, Time = 4821.45 ms, Matches = 26, Positions: 954638 3972674 3972675 5111384 5111385 ...
Thread 0: called

Threads = 2, Time = 2338.52 ms, Matches = 26, Positions: 954638 3972674 3972675 5111384 5111385 ...
Thread 0: called
Thread 1: called

Threads = 4, Time = 3219.66 ms, Matches = 26, Positions: 954638 3972674 3972675 5111384 5111385 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 2345.51 ms, Matches = 26, Positions: 954638 3972674 3972675 5111384 5111385 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

```
chenih@Chenih5:~/Parallel Algorithms/Lab2$ ./approx_match 8 100000000 2
Fixed pattern: "cxhoyrtb"

Threads = 1, Time = 8776.05 ms, Matches = 22, Positions: 1509215 1634537 13247356 13247357 13247358 ...
Thread 0: called

Threads = 2, Time = 4533.23 ms, Matches = 22, Positions: 1509215 1634537 13247356 13247357 13247358 ...
Thread 0: called
Thread 1: called

Threads = 4, Time = 4447.40 ms, Matches = 22, Positions: 1509215 1634537 13247356 13247357 13247358 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called

Threads = 8, Time = 4526.36 ms, Matches = 22, Positions: 1509215 1634537 13247356 13247357 13247358 ...
Thread 0: called
Thread 1: called
Thread 2: called
Thread 3: called
Thread 4: called
Thread 5: called
Thread 6: called
Thread 7: called
```

(3). 运行时间

从 KMP 实验结果表明，随着主串长度的增加，程序运行时间基本呈线性增长，符合 KMP 算法的时间复杂度预期。在数据规模较小时，多线程并未体现出明显的加速效果，反而由于线程调度与边界处理的开销，导致运行时间相比单线程略有上升，在小规模问题中并行化并不划算。随着主串长度增长到 10^6 至 10^7 级别，2 至 4 个线程开始展现出一定加速效果，但 8 线程在部分情况下加速不稳定，说明并行调度成本仍是限制因素。而在 10^8 级别的大规模数据下，8 线程实现了超过 5 倍的加速，并行效率可以被充分释放。

相比之下，近似匹配算法的计算密集度更高，其运行时间在单线程下随文本长度上升得更为迅速，表明该算法在每一位置都需进行大量的编辑距离运算。在小规模文本数据下（如 10^5），多线程基本无效，甚至会因线程开销增加而变慢。随着文本进入中等规模后到 10^6 及以上，2 至 8 线程均能带来不同程度的加速，特别在 10^7 以上的数据中，8 线程通常可实现接近或超过 2 倍的性能提升，说明该算法更适合在任务量大、计算密集的环境中进行并行化处理。

(4). 加速比计算

加速比计算公式：$Speedup = \frac{T_{single}}{T_{paralle}}$。

(i). 并行 KMP 算法：

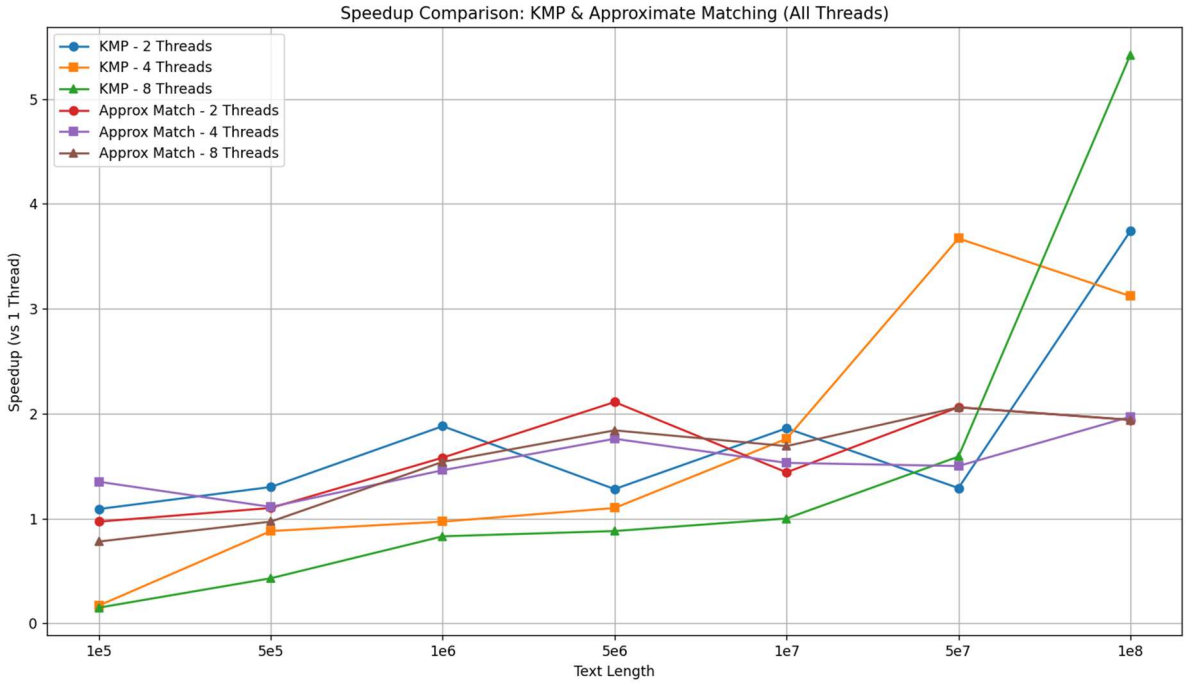
数据规模\线程数	1	2	4	8
100,000	1.00	1.09	0.17	0.15
500,000	1.00	1.30	0.88	0.43
1,000,000	1.00	1.88	0.97	0.83
5,000,000	1.00	1.28	1.10	0.88
10,000,000	1.00	1.86	1.76	1.00
50,000,000	1.00	1.29	3.67	1.59
100,000,000	1.00	3.74	3.12	5.42

(ii). 近似匹配算法：

数据规模\线程数	1	2	4	8
100,000	1.00	0.97	1.35	0.78
500,000	1.00	1.10	1.11	0.97
1,000,000	1.00	1.58	1.46	1.54

5,000,000	1.00	2.11	1.76	1.84
10,000,000	1.00	1.44	1.53	1.69
50,000,000	1.00	2.06	1.50	2.06
100,000,000	1.00	1.94	1.97	1.94

加速比曲线：



从图中可以看出，KMP 算法的曲线起伏明显，特别是在线程数较多（如 4 和 8）时，在小规模数据下出现明显下跌，加速比低于 1，甚至低至 0.15，表明并行化反而造成了负优化。而到了大数据点（如 10^8），KMP 的 8 线程加速比陡然上升，说明其并行性能只有在任务量足够大时才被激发出来。这条曲线呈现出“前低后高”的非线性特征，且波动剧烈。

与之相比，近似匹配的加速比曲线更加平稳。2、4、8 线程的加速比整体走势基本一致，随着文本规模增加呈现缓慢上升，虽然也存在波动，但幅度较小，整体表现出“稳中有升”的趋势。特别是在中等规模之后，各曲线保持在 1.5～2.0 区间，说明该算法更容易保持并行效率。

从图形轮廓上可以看出，KMP 的并行性能依赖大数据驱动，而近似匹配的加速比表现更均匀、更可控，适合在不同规模下使用多线程进行优化。

5. 分析与总结

(1). 负载均衡分析

KMP 算法本身具有线性时间复杂度，理论上将主串均匀划分至各线程可实现较好的负载平衡。然而，出于匹配完整性的要求，每个线程必须在其任务起点前额外保留 m-1 个字符的重叠区用于边界判断，导致每个线程的实际处理长度存在偏差。此外，部分线程在匹配过程中可能遇到较多的失败回

溯，进一步放大了处理时间的差异，尤其在线程数增多、数据规模较小时，这种边界冗余对负载均衡的不利影响更加明显。

相比之下，近似匹配算法由于在主串每一位置都需进行编辑距离的完整计算，其任务具有高度的局部独立性。即使采用静态划分，每个线程的计算负载也相对均匀，且不涉及跨段依赖或状态传递，负载均衡表现更加稳定。但需注意的是，当 k 值较大时，单个窗口内的计算量波动会有所增加。

综上，在本实验中，KMP 并行版本的负载均衡受制于边界重叠处理与状态依赖问题，而近似匹配算法因计算独立性强，在并行环境下展现出更优的负载均衡性能。

(2). 算法对比分析

KMP 算法的主要优势是在于利用前缀函数在匹配过程中避免重复回溯，算法在串行情形下已经具备较高的效率。然而，这种线性流程的结构决定了它并不适合并行化，当线程数为 4 或 8 时，加速比出现显著下降，这种现象主要来源于任务划分粒度不足、线程启动与同步带来的额外开销，以及模式串可能跨越分段边界所引发的重复计算。在规模增大到 10^7 及以上时，KMP 的并行效率才逐步体现出来，在 10^8 的主串长度下，8 线程带来了超过 5 倍的加速比，表明其并行性能依赖于数据量的驱动，在任务足够大的前提下，KMP 仍可获得显著的时间收益。

近似匹配算法由于需要在每一个滑动窗口位置上计算编辑距离，其计算密集度远高于 KMP 算法，当允许的最大编辑距离 k 为正时，子串间的比较不再是简单的字符匹配，而是涉及到动态规划矩阵的计算。这种高度局部化、独立性的计算特征使其非常适合并行化处理。在运行结果中可以看到，近似匹配算法在中等数据规模（如 10^6 至 10^7）下，多线程均能带来可观的加速比，能有效降低总运行时间。此外，算法的加速比曲线波动较小，表现出良好的负载均衡性和扩展性，可以在计算密集型任务上能够更加充分利用多核资源。

综合来看，KMP 是一种低计算、高依赖的算法，在并行化过程中需要较大的任务粒度才能有效分摊调度开销，更适合串行执行或配合其他策略进行块级划分优化。而近似匹配算法则属于高计算密集型任务，在多线程环境下表现出更高的稳定性与扩展能力，对大规模、容错要求较高的文本匹配任务，近似匹配算法的并行版本更佳。

(3). 适应性

KMP 算法在小规模文本数据下表现较好，线性时间复杂度使其在串行模式下具有极高的效率。然而其并行性能依赖于任务粒度的放大，实验中也观察到，只有当主串长度达到大规模级别时，多线程才展现出明显加速，线程数多时若任务划分不均或数据不足，会导致资源浪费和性能下降，在小数据下则因调度与划分开销反而拖慢速度。因此，KMP 对输入规模较为敏感，适应性较弱，KMP 更适合部署在高主频、低并发的计算平台。

而近似匹配算法在各类数据规模下表现得更加平稳，每个匹配位置计算完全独立，线程间几乎无依赖，资源利用率更高，调度更加平滑，具有更强的鲁棒性，有稳定的加速比，说明其适应性更强，不仅在多核 CPU 上表现优异，也非常适合迁移到 GPU、并行集群等平台进行加速。

在误差适应性方面，KMP 仅适用于精确匹配场景，对字符级误差完全不容忍。而近似匹配算法原生支持编辑距离容忍，可以灵活处理模糊匹配问题，适应性明显更高。此外，KMP 需特别处理分段边界和模式串重叠问题，增加了与底层硬件交互的复杂性，而近似匹配任务划分对硬件透明，部署更灵活。

因此，KMP 算法在精确匹配任务中具备良好的串行效率，但其对输入规模、线程划分和硬件平台要求较高，适应性相对有限；而近似匹配算法在多维度上表现出更强的稳定性与通用性，尤其在多核并行与容错匹配任务中展现出良好的扩展能力和硬件适配能力。

(4). 性能分析与影响因素

从算法本身的结构特性来看，KMP 属于典型的线性复杂度算法，串行执行效率极高，但并行化时需考虑子串重叠、边界一致性等问题，导致线程之间存在一定的依赖与同步需求，降低了并行效率。而近似匹配算法每个起始位置的判断完全独立，适合数据并行，不存在串间依赖，使其在多线程环境下能更充分发挥并行计算能力。

其次，线程调度与任务划分策略对性能影响显著。实验中采用的分块策略将主串按区间划分给各线程，在模式串较短时易于实现，但若未考虑模式串长度造成的边界重叠问题，可能会导致漏匹配或重复匹配。此外，当主串长度较小时，划分出的任务粒度过小，线程调度开销占据较大比例，甚至导致性能下降。因此，任务粒度与线程数量之间的匹配程度是决定性能的关键因素。

内存访问局部性与缓存一致性也会对程序性能产生影响。KMP 算法在匹配过程中需频繁访问前缀函数表和主串数据，如果多个线程处理相邻数据，可能出现缓存行竞争；而近似匹配中的动态规划矩阵更新具有更强的内存写入负载，对带宽的依赖性更强。当线程数增加时，主存访问压力上升，可能成为瓶颈，限制加速比的提升。

此外，数据规模对性能发挥也十分重要。从实验结果来看，只有在主串达到一定长度（如 10^7 以上）时，并行线程的开销才会被足够大的计算量所掩盖，从而实现显著加速。对于小规模数据，串行算法因其无调度、无开销特性，反而具备更强的性能优势。硬件资源的利用率也影响最终性能，需合理控制线程数量、避免资源过度争用。

综上，算法的运行效率不仅取决于理论复杂度，更受到任务划分策略、线程调度成本、内存访问模式、数据规模大小以及硬件资源分配等多重因素的综合影响。在实际部署并行字符串匹配算法时，应根据应用场景有针对性地调整线程数、优化数据划分方式，并根据平台特性选择合适的算法结构，以获得最佳的性能表现。

(5). 改进方案

对于 KMP 算法，可以尝试引入边界状态共享机制，即每个线程在启动时预加载其任务起点对应的前缀函数状态，从而避免因忽略上下文而重复匹配边界区域。这不仅减少了边界冗余计算，也提升了匹配连续性。进一步地，可结合动态任务划分策略，由主线程维护任务队列，多个线程按需拉取匹配段，避免线程因处理负载不均而早退，提升整体核利用率。

近似匹配算法方面，计算密度虽高但存在大量可剪枝空间。可引入早停机制（如编辑距离超过设定阈值时立即跳出），降低无效计算开销。此外，当前划分策略基于静态等长划分，建议结合负载感知调度策略，根据主串局部复杂度预估匹配计算成本，进行非均匀分段，使任务划分更贴近真实计算负载。

在系统层面，针对高线程数情况下调度与缓存竞争问题，可使用 OpenMP 中的 guided 或 dynamic 调度策略，实现负载自适应。同时，对于大规模文本匹配任务，建议将近似匹配迁移至 GPU 或其他加速平台，以发挥其在高并发、高计算强度任务中的并行优势。

6. 实验感想

通过本次的编程实验，我不仅深入理解了 KMP 算法与近似匹配这两种字符串匹配算法的原理和实现方式，还学习到串行算法向并行化迁移过程中所面临的挑战。实验过程让我认识到，并行不一定总是带来更快的效果，像线程调度、任务划分、边界处理这些细节处理不好，反而可能让程序变慢，尤其是在数据量比较小时这种情况更明显。

在实验的过程中，我对多线程的编程方式有了更具体的认识，也学会了如何结合算法特点去设计合理的并行策略。两个算法在不同线程数和数据规模下的表现差异让我对算法适用的场景有了更清晰的理解。在绘制加速比曲线并进行负载分析的过程，更让我认识到实验数据背后隐藏的规律和问题是需要认真思考和总结的。

本次实验不仅提升了我在算法实现、性能测试和结果分析方面的能力，也培养了我在并行编程中的细致思维和系统优化意识，是一次非常有价值的实践学习体验。

7. 参考资料

1. https://blog.csdn.net/JacksonLiang/article/details/1285732?ops_request_misc=%257B%2522request%255Fid%2522%253A%25227fa21faee2bc7c5d3217e8a5e40ef002%2522%252C%2522scm%2522%253A%252220140713.130102334..%2522%257D&request_id=7fa21faee2bc7c5d3217e8a5e40ef002&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduend~default-1-1285732-null-null.142^v102^pc_search_result_base5&utm_term=%E5%AD%97%E7%AC%A6%E4%B8%B2%E8%BF%91%E4%BC%BC%E5%8C%B9%E9%85%8D%E7%AE%97%E6%B3%95&spm=1018.2226.3001.4187